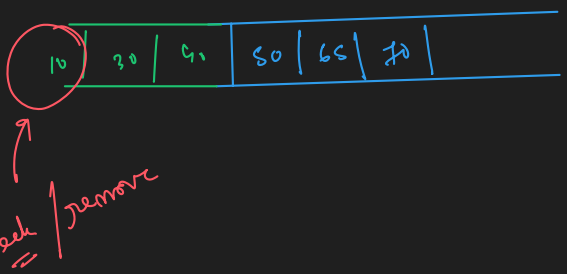
Day - 42

Heap or Priority Queue

max min (default)

min PQ

50 65 70 10 30 40



Peek / remove

- data will insert or remove based on priority.

add() → O(log n)
remove() → O(log n)
peek() → O(1)

using JCF

```
PriorityQueue<Integer> PQ = new PriorityQueue<>(); // asc. order.
PriorityQueue<Integer> PQ = new PriorityQueue<>(Comparator.reverseOrder()); // desc order.
```

this → S1.

```
int compareTo(Student s2)
return this.rank - s2.rank;
}
```

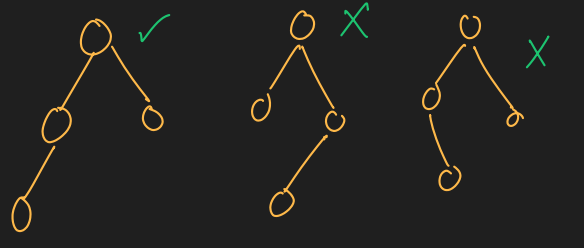
this
Student S1 = ('Priyam', 10)
S2 = ('Dhanush', 15) ✓

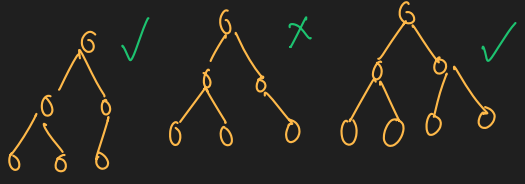
S1 S2 → -ve S1 < S2
+ve S1 > S2
= S1 == S2

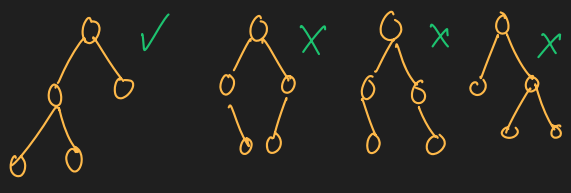
Heap → max [max ele ↑ . Priority ↑]
→ min (default) [min ele ↓ Priority ↑]

→ visualized → BT (complete BT).
→ implemented as array or AL.

Property ① Complete BT
[CBT is a BT which levels are completely filled except possibly the last one, which is filled from left to right]

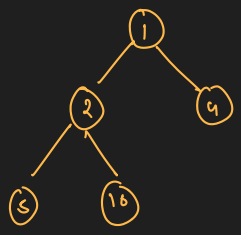






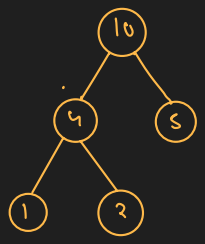
② for min heap

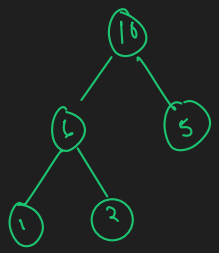
Children > Parent



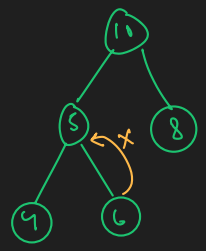
for max heap

Children < Parent



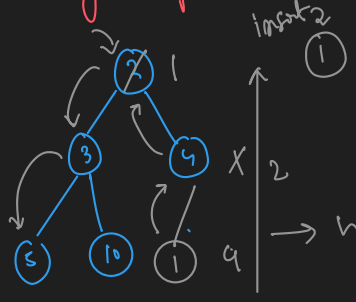


[max heap]

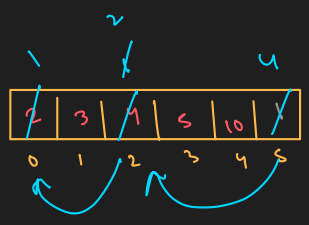


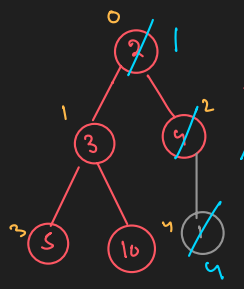
[not a heap]

why heap is not implemented a tree



Implement as Array or AL





Parent → Child rel"

parent (i)

child = 2i+1

= 2i+2

p = i = 0	p = i = 1
c1 = 1	c1 = 3
c2 = 2	c2 = 4

Child → Parent rel"

child = x

parent = (x-1)/2

insert (1) 5-1/2 = 2, 2-1/2 = 0

insert(ele) - O(log n)

to make a heap → O(n log n)

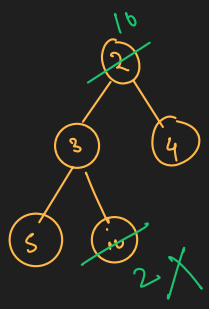
Step 1 add at last position in AL / Array

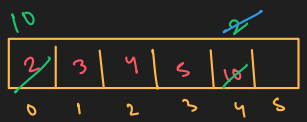
Step 2 adjust the heap.

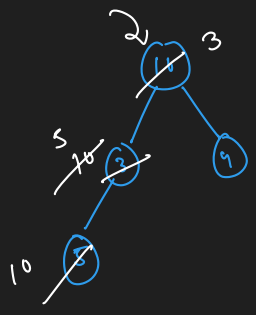
```
while ( child val < parent val ) {
    swap ( child, parent ) ;
}
```

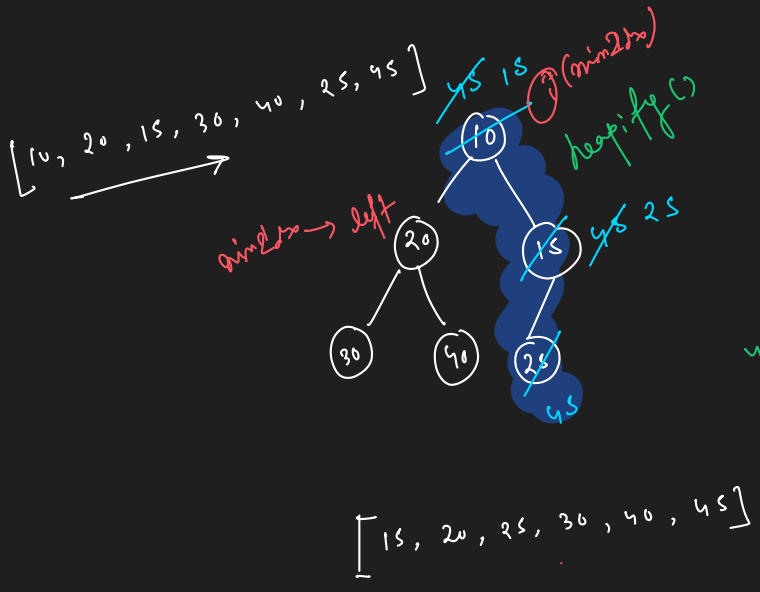
```java
// add()
public void add(int data){
    arr.add(data);

    // adjust the heap
    int c = arr.size()-1;
    int p = (c-1)/2;

    while (arr.get(c) < arr.get(p)) {
        // swap
        int temp = arr.get(c);
        arr.set(c, arr.get(p));
        arr.set(p, temp);

        // update
        c = p;
        p = (c-1)/2;
    }
}
```

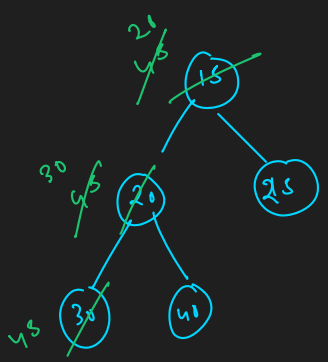
Remove

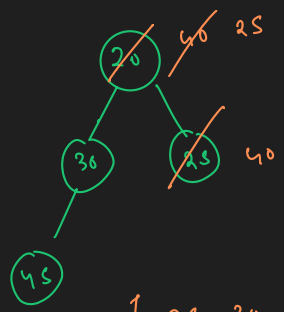












[15, 20, 25, 30, 40, 45]

[20, 30, 25, 45, 40]

[25, 30, 40, 45]

Step ① swap first and last position

Step ② remove the value of the last idx

Step ③ fix the heap using heapify() .

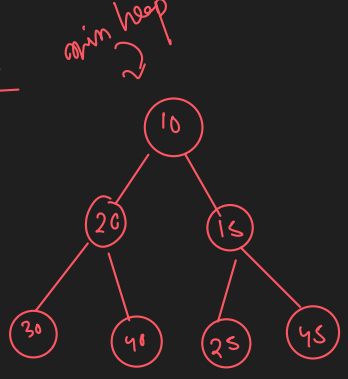
```java
// remove()
public int remove(){
    // step 1
    int temp = arr.get(index: 0);
    arr.set(index: 0, arr.get(arr.size()-1));
    arr.set(arr.size()-1, temp);

    // step 2
    int data = arr.remove(arr.size()-1);

    // step 3
    heapify(i: 0);

    return data;
}
```

```java
public void heapify(int i){
    int left = 2*i+1;
    int right = 2*i+2;

    int minIdx = i;

    if(left<arr.size() && arr.get(left) < arr.get(minIdx)){
        minIdx = left;
    }
    if(right <arr.size() && arr.get(right) < arr.get(minIdx)){
        minIdx = right;
    }

    // swap
    if(minIdx != i){
        int temp = arr.get(i);
        arr.set(i, arr.get(minIdx));
        arr.set(minIdx, temp);

        heapify(minIdx);
    }
}
```

H.W Max Heap

day-43

Heap Sort



Asc-order sort → Max heap

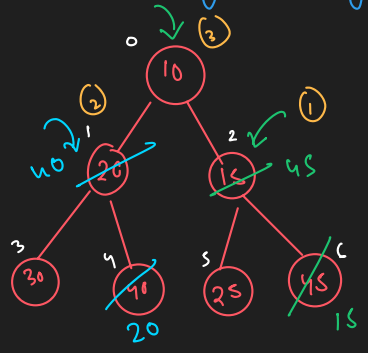
Dose-order sorting → Min heap (H.W)

heapify → top down approach

10	20	15	30	40	25	45
0	1	2	3	4	5	6

(1)

I want to make the given array as Max heap



[Par → child]

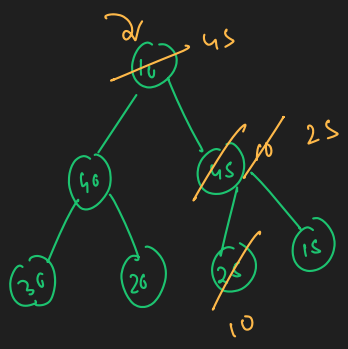
Par = i

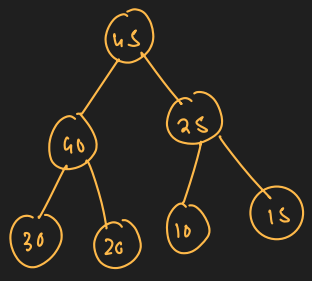
child = 2i+1, 2i+2

child → Par
↓ ↓
x (x-1)/2

Max heap par > child.

Build





(Max heap)

for each heapify → log n

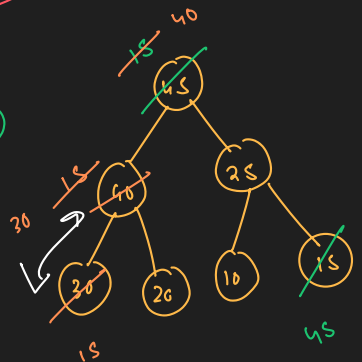
n/2 ele gets arranged

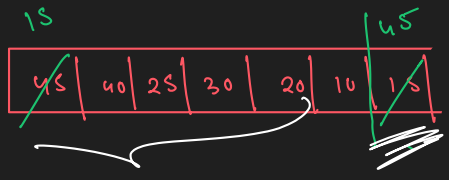
$\therefore T.C = T\left(\frac{n}{2}\log n\right)$

$= O(n \log n)$

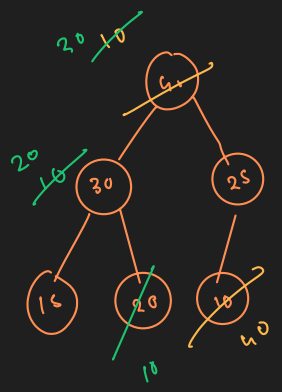
Sorting (asc)

(1)

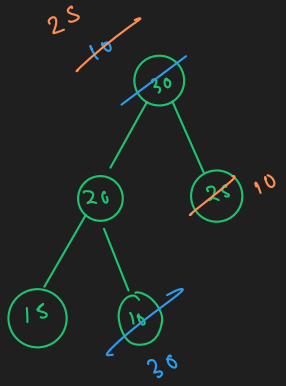


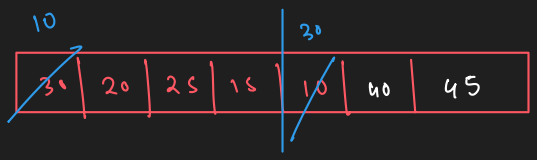


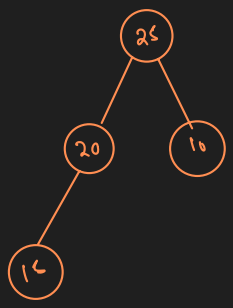
(1)

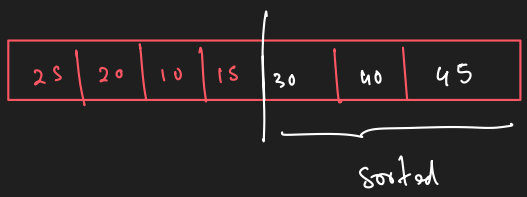


```
public static void heapify(int arr[], int i, int size){
    int left = 2*i+1;
    int right = 2*i+2;

    int maxIdx = i;
    if(left<size && arr[left]>arr[maxIdx]){
        maxIdx = left;
    }
    if(right<size && arr[right]>arr[maxIdx]){
        maxIdx = right;
    }

    // swap
    if(maxIdx != i){
        int temp = arr[i];
        arr[i] = arr[maxIdx];
        arr[maxIdx] = temp;

        heapify(arr, maxIdx, size);
    }
}
```









```
public static void heapsort(int arr[]){
    // step 1 - max heap
    for(int i=(arr.length/2); i>=0; i--){
        heapify(arr, i, arr.length);
    }
    // sorting
    for(int n=(arr.length-1); n>=1; n--){
        // swap
        int temp = arr[0];
        arr[0] = arr[n];
        arr[n] = temp;

        heapify(arr, i: 0, n);
    }
}
```

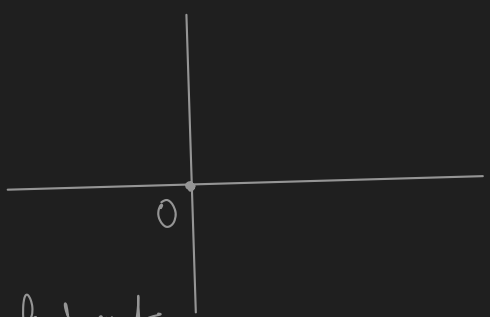
Questions

Nearby Car

$C_1(3,3)$

$C_2(5,-1)$

$C_3(-2,4)$



From origin find out nearby k cars

for, k = 2 Ans: C_1, C_3

```java
static class Car implements Comparable<Car>{
    int x;
    int y;
    int dist;

    Car(int x, int y){
        this.x = x;
        this.y = y;
        this.dist = x*x + y*y;
    }

    @Override
    public int compareTo(Car c2){
        return this.dist - c2.dist;
    }
}

Run | Debug
public static void main(String[] args) {
    int[][] arr = {{3, 3}, {5, -1}, {-2, 4}};
    int k = 2;

    PriorityQueue<Car> cars = new PriorityQueue<>(); // min heap
    for(int i=0; i<arr.length; i++){
        cars.add(new Car(arr[i][0], arr[i][1]));
    }

    for(int i=0; i<k; i++){
        System.out.println(cars.peek().x+", "+cars.peek().y);
        cars.remove();
    }
}
```

Question 2

Given N ropes of diff length. Task is to connect these ropes into one rope with min cost such that, the cost to connect two ropes is equal to sum of their lenght

ropes = {2, 3, 3, 4, 6} Ans : 41

cost = 5 {3, 4, 5, 6}

cost = 7 {5, 6, 7}

cost = 11 {7, 11}

cost = 18

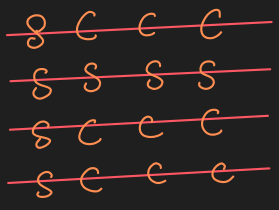
Total Cost = 5 + 7 + 11 + 18 = 41

```java
int ropes[] = {2, 3, 3, 4, 6};

PriorityQueue<Integer> pq = new PriorityQueue<>();
for(int i=0; i<ropes.length; i++){
    pq.add(ropes[i]);
}

int cost = 0;
while (pq.size() != 1) {
    int len = pq.remove()+pq.remove();
    pq.add(len);
    cost += len;
}
System.out.println(cost);
```

Day-44

Question 3 : Weakest Soldiers , Civilian = 0

S = 1, C



weakest

1 0 0 0 ① ✓

1 1 1 1 ④

1 0 0 0 ② ✓

1 0 0 0 ③

row → powerfull no. of soldier 1.

if for two rows no of soldier same then based on idx value

for greater idx stronger the row.

find k- weakest row.

k = 2 Ans : R0, R2

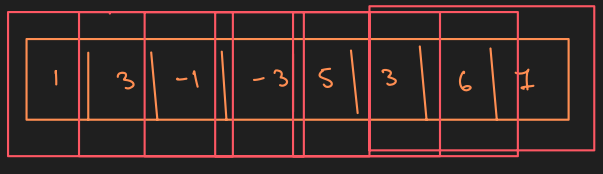
```
static class RowInfo implements Comparable<RowInfo>{

    int count;
    int idx;

    public RowInfo(int count, int idx){
        this.count = count;
        this.idx = idx;
    }

    @Override
    public int compareTo(RowInfo r2){
        if(this.count == r2.count){
            return this.idx - r2.idx;
        }
        return this.count - r2.count;
    }
}
```

```
public static void main(String[] args) {

    int [][]rows = {
        {1, 0, 0, 0},
        {1, 1, 1, 1},
        {1, 0, 0, 0},
        {1, 0, 0, 0},
    };

    int k = 2;

    PriorityQueue<RowInfo> rowInfos = new PriorityQueue<>();

    for(int i=0; i<rows.length; i++){
        int c = 0;
        for(int j=0; j<rows[0].length; j++){
            c += rows[i][j];
        }
        rowInfos.add(new RowInfo(c, i));
    }

    for(int i=0; i<k; i++){
        System.out.println("R"+rowInfos.remove().idx);
    }
}
```

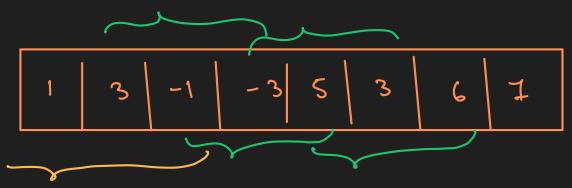
Question 4 Sliding Window Maximum

find Maximum of all subarrays of size k

[1 3 -1 -3 5 3 6 7], K = 3 Ans : 3, 3, 5, 5, 6, 7



Ans: Max = 3, 3, 5, 5, 6, 7



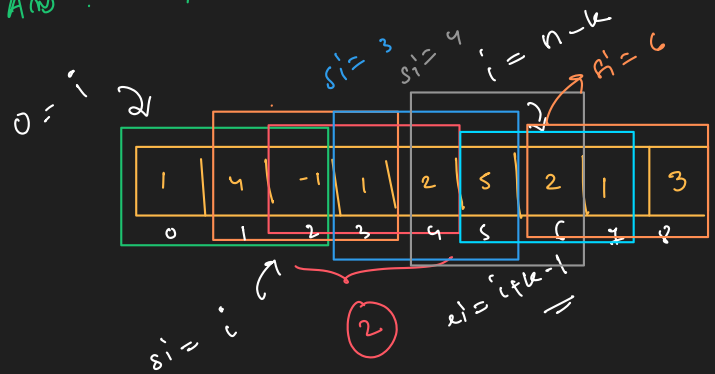
pq → 7 | 6 | 5 | 3 | 3 | 1 | -1 | -3

max heap

si = 3 idx = 4

Ans : 4 , 4 , 2 , 5 , 5 , 5 , 3

n = arr.length



Ans : 3 , 3 , 5 , 5 , 6 , 7

idx = 5

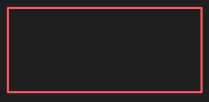
pq → 5 | 3 | 2 | 2 | 1 | 1 | 1 | -1

(max heap)

top.idx < si

Ans : 4 , 4 , 2 , 5 top → remove

5 , 5 , 3



si = i

ei = i+k-1

pq2

top → data

&

value, idx

while (top.idx < si of window) {

pq.remove() ;

}

```
int[] arr = {1, 4, -1, 1, 2, 5, 2 , 1, 3 };
int k = 3;

PriorityQueue<Info> pq = new PriorityQueue<>();
int max[] = new int[arr.length-k+1];

for(int i=0; i<k; i++){
    pq.add(new Info(arr[i], i));
    max[0] = pq.peek().data;
}

for(int si=1; si<max.length; si++){
    int ei=si+k-1;
    pq.add(new Info(arr[ei], ei));

    while (pq.peek().idx < si) {
        pq.remove();
    }
    max[si] = pq.peek().data;
}

for(int i=0; i<max.length; i++){
    System.out.print(max[i]+" ");
}
System.out.println();
```

```
static class Info implements Comparable<Info>{

    int data;
    int idx;

    Info(int data, int idx){
        this.data = data;
        this.idx = idx;
    }

    public int compareTo(Info i2){
        return i2.data - this.data;
    }
}
```

Another Approach using Array list

```
int[] arr = {1, 4, -1, 1, 2, 5, 2 , 1, 3 };
int k = 3;

PriorityQueue<Info> pq = new PriorityQueue<>();
ArrayList<Integer> max = new ArrayList<>();

for(int i=0; i<k; i++){
    pq.add(new Info(arr[i], i));
}
max.add(pq.peek().data);

for(int si=1; si<arr.length-k+1; si++){
    int ei=si+k-1;
    pq.add(new Info(arr[ei], ei));

    while (pq.peek().idx < si) {
        pq.remove();
    }
    max.add(pq.peek().data);
}
System.out.println(max);
```